Links, URLs, and navigation

Links are what turn a collection of independent files into a navigable site, and they are how the web ties itself together. This lesson is about the `<a>` element and the URLs that go inside it.

The anchor element

A link in HTML is written with the `<a>` element. The address you want to link to goes in the `href` attribute, and the visible text goes between the opening and closing tags.

```html
<a href="https://example.com">Example site</a>
```

When the user clicks the link, the browser navigates to whatever the `href` points to. The default behavior is to replace the current tab with the new page.

The `<a>` element can also wrap images, headings, or other content. Anything inside the tags becomes clickable. Do not put another link or a button inside a link; nested interactive elements are confusing for browsers and assistive tools.

```html
<a href="/about">
  <img src="team.jpg" alt="The Acme team">
</a>
```

Use links for navigation: going to another page, another site, or another place on the same page. Use `button` tag (covered in later lessons) for actions that happen on the current page, such as opening a menu, submitting a form, or dismissing a dialog. A link with `href="#"` just to catch a click is usually a sign that you wanted a button.

Absolute, root-relative, and relative URLs

The value of `href` can take a few different shapes depending on where the link is going.

Absolute URLs include the full protocol and host. Use them when linking to a different site.

```html
<a href="https://example.com/articles/intro">External article</a>
```

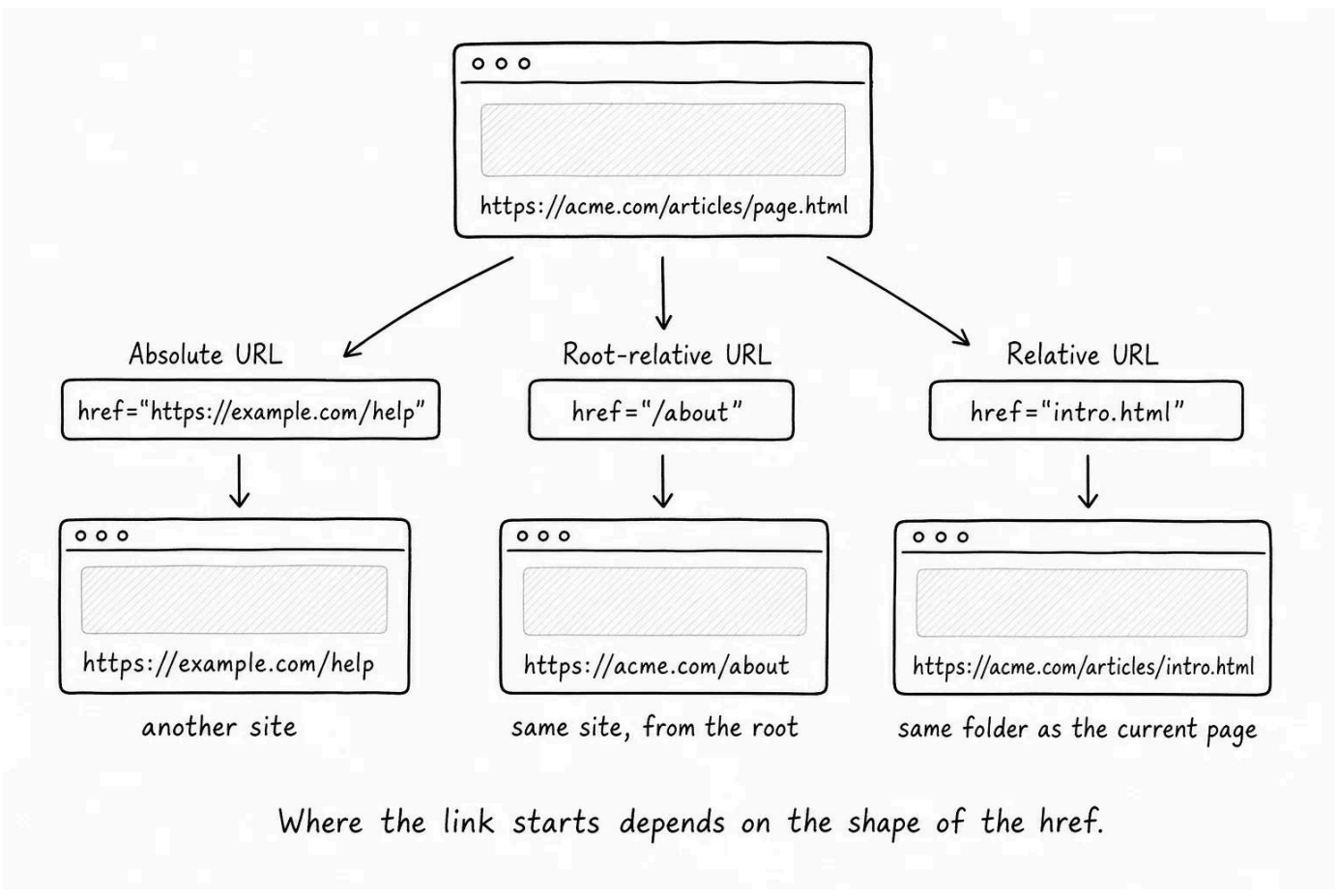
Root-relative URLs start with `/` and are interpreted from the root of the current site. Use them for internal links across an app.

```html
<a href="/articles/intro">Intro article</a>
```

A link with `href="/articles/intro"` from `https://acme.com/about` points to `https://acme.com/articles/intro`.

Relative URLs are interpreted from the current page's location. Use them when linking between sibling files.

```html
<a href="about.html">About</a>
<a href="articles/intro.html">Intro article</a>
<a href="../index.html">Back to home</a>
```



For real apps, **root-relative URLs** are usually the right default for in-app links. They keep working when you move a page, and they do not depend on the current page's location.

Fragment links

A URL can include a `#` followed by an id. The browser uses that to jump to an element with the matching id on the page.

```html
<a href="#pricing">Jump to pricing</a>
```

```
...
...
...
<h2 id="pricing">Pricing</h2>
```

Clicking the link scrolls the page to the `Pricing` heading. Fragment links work inside the same page or across pages:

html

```
<a href="/about#team">Meet the team</a>
```

That link goes to `/about` and scrolls to the element with `id="team"`.

Opening links in new tabs

By default, a link replaces the current tab. To open the link in a new tab, set `target="_blank"`.

html

```
<a href="https://example.com" target="_blank" rel="noopener">External site</a>
```

The `rel="noopener"` attribute is good practice whenever you use `target="_blank"`. It prevents the new page from getting a JavaScript reference back to the page that opened it, which is a small but real security and performance improvement. Modern browsers add this behavior automatically, but writing it is still the convention. You will learn more about this in the future lessons.

You may also see `rel="noreferrer"`, which goes further by hiding the referring URL from the destination. Use it when you do not want the destination to know where the click came from.

Default to same-tab - Open links in the same tab unless there is a real reason not to. Users can always open a new tab themselves; forcing it removes that choice and takes the click out of the normal back-button flow.

Link text matters

A link is a navigation choice. The text inside the link should tell the user where it goes.

Avoid generic text:

html

```
<a href="/pricing">click here</a> to see our plans.
```

Prefer text that describes the destination:

```html
See our <a href="/pricing">pricing plans</a>.
```

Why it matters:

- **Screen readers** can list every link on a page. A list of "click here, click here, click here" is useless.
- **Search engines** use link text as a signal for what the destination is about.
- **Scanning** the page is easier when link text describes where each link goes.

If a link contains only an image, the image's `alt` text becomes the link text for screen reader users. That is one more reason useful alt text matters.

mailto and tel links

A few special URL schemes start a different kind of action when clicked. `mailto:` opens the user's email client; `tel:` starts a phone call on capable devices.

```html
<a href="mailto:hello@example.com">Email us</a>
<a href="tel:+12025550123">Call us</a>
```

Both are useful as shortcuts, but they depend on the user's device having an email or phone app configured. Do not rely on them as the only contact path.

When URLs come from app code

URL structure is one of the places where HTML, routing, and server code meet.

A few small habits pay off later:

- **Use root-relative URLs** for in-app links in most apps. Hardcoding `https://yourapp.com/about` breaks in local development, staging, and preview environments.
- **Generate URLs from your router** instead of building them by hand in templates (i.e. if your backend code handles the page routing). Renaming a route then updates every link automatically.

- **Pick stable URL paths**. Once a link is out in the world, breaking it is expensive. Bookmarks, emails, and search results all point at it.

Try it

Make a two-page site by creating two files in the same folder.

`index.html`:

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Home</title>
  </head>
  <body>
    <h1>Home</h1>
    <p>This is the homepage. Read more <a href="about.html">about this site</a>.</p>
  </body>
</html>
```

`about.html`:

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>About</title>
  </head>
  <body>
    <h1 id="top">About</h1>
    <p>This page tells you about the site.</p>
    <h2 id="contact">Contact</h2>
    <p>Get in touch at <a href="mailto:you@example.com">you@example.com</a>.</p>
    <p><a href="#top">Back to top</a> · <a href="index.html">Back to home</a></p>
  </body>
</html>
```

Open `index.html` and click the link. On the about page, click the fragment link, then "back to home", and watch the URL change in the address bar. You are seeing the two most common local-file links: `about.html` points to another file in the same folder, and `#top` points to an element on the current page.

What is the href attribute?

Click to Reveal the Answer

Already Know that | Didn't Know that | Skip Question

LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson